

AWS ECR

Elastic Container Registry

Concept Overview:

Introduction of ECR	1
Why use ECR	2
Features of ECR	3
Types of ECR	4
Lifecycle of ECR Image	5
Prerequisites Before Hands-on	6
Hands-on	7

Introduction of ECR:

Amazon Elastic Container Registry (Amazon ECR) is an AWS managed container image registry service that is secure, scalable, and reliable.

Amazon ECR supports private repositories with resource-based permissions using AWS IAM. This is so that specified users or Amazon EC2 instances can access your container repositories and images.

You can use your preferred CLI to push, pull, and manage Docker images, Open Container Initiative (OCI) images, and OCI compatible artifacts.

Why use ECR:

- Store Docker and OCI container images securely.
- Works natively with ECS, EKS, Lambda, and CI/CD pipelines.
- Image scanning, encryption, IAM access control, and managed signing.
- Fully managed, highly available, and scales automatically.
- Automatically remove old or unused images with lifecycle policies.
- Replicate images across AWS Regions and accounts.
- Pull-through cache reduces latency and dependency on external registries.
- No infrastructure to manage or maintain.

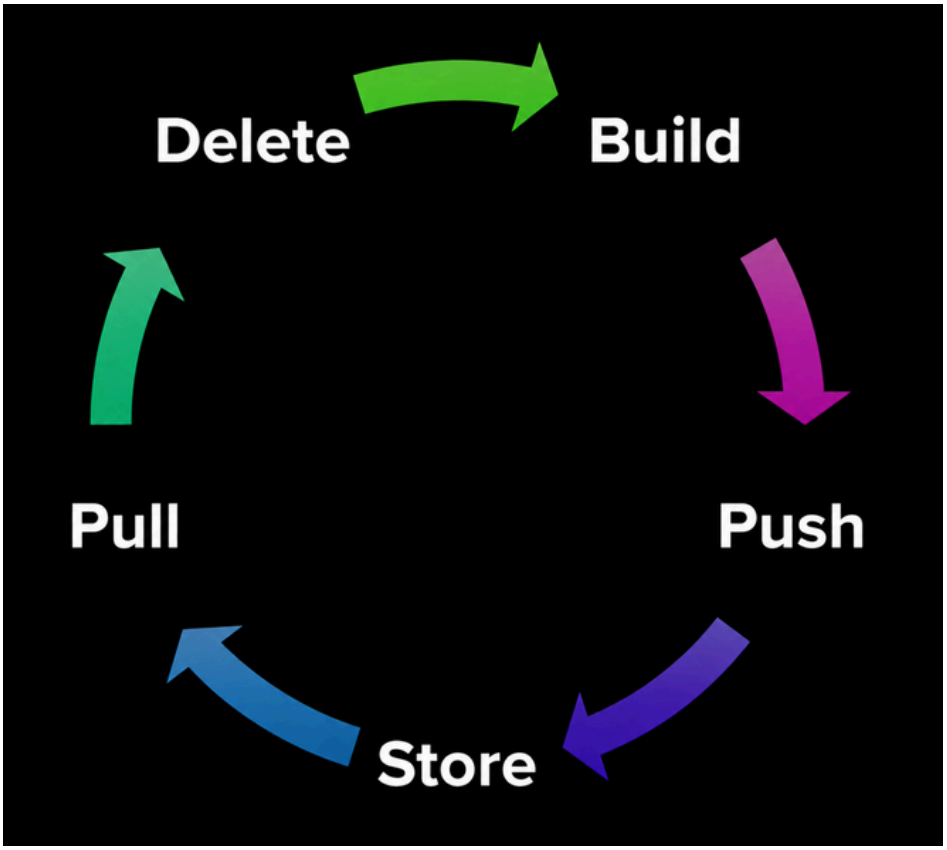
Features of ECR:

- Lifecycle policies: Automatically clean up unused images.
- Image scanning: Detect vulnerabilities in container images.
- Cross-Region & cross-account replication – Replicate images across Regions and AWS accounts.
- Pull through cache: Cache images from upstream registries for faster access.
- Repository creation templates: Apply predefined settings to automatically created repositories.
- Managed signing: Automatically sign container images with cryptographic signatures for integrity and authenticity.

Types of ECR:

- Private ECR: Securely stores container images within your AWS account. Uses IAM authentication and supports fine-grained access control for internal and production workloads.
- Public ECR: Hosts container images that anyone can pull without AWS authentication, making it ideal for open-source and publicly shared images.

Lifecycle of ECR Image:



Build: Create a new Docker image from your application's source code and Dockerfile, packaging all required files, dependencies, and configurations into a portable container image ready for deployment.

Push: Upload the newly built Docker image from your local machine to an Amazon ECR repository. This makes the image available for deployment, sharing with team members, and use in CI/CD pipelines.

Lifecycle of ECR Image:

Store: Amazon ECR securely stores your container images, manages version tags, controls access through IAM, and can automatically scan images for vulnerabilities and remove outdated images using lifecycle policies.

Pull: Download a Docker image from Amazon ECR to a local machine, EC2 instance, ECS service, EKS cluster, or other deployment environment so it can be used to run containers or deploy applications.

Delete: Remove old or unused Docker images from your local machine to free up disk space and prevent confusion between outdated and current versions. This cleanup only affects your local environment and does not delete images stored in Amazon ECR.

Prerequisites Before Hands-on:

- AWS CLI installed and configured (aws configure) with valid Access Key ID, Secret Access Key, Region, and Output format.
- Docker Desktop (Windows/macOS) or Docker Engine (Linux) installed and running properly.
- Basic understanding of Docker concepts, including images, containers, and Dockerfiles.
- A sample application with a Dockerfile ready to build a container image.
- An Amazon ECR Repository created (or permission to create one during the lab).
- Basic knowledge of Linux terminal or Command Prompt/PowerShell commands.

Hands-on lab:

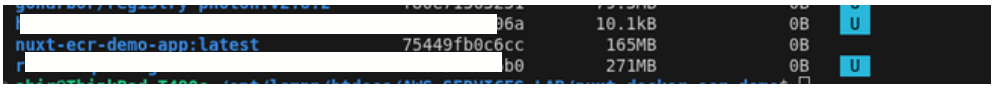
In my case, I'm using Linux Ubuntu Destro.
Here is the repo link which i use.
repo: https://github.com/alamgirweb11/nuxt-docker-ecr-demo

Step 1:
Build a docker image of this project.
run this command:

```
docker build -t nuxt-ecr-demo-app .
```

To see this image run bellow command.

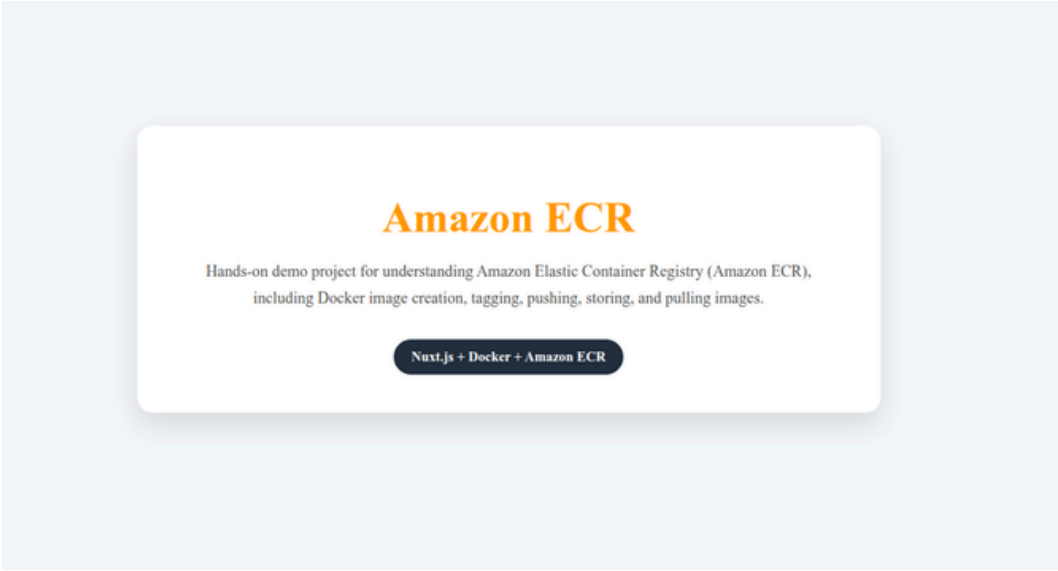
```
docker images
```



Now run the image

```
docker run -p 3000:3000 nuxt-ecr-demo-app
```

Base url return this demo page

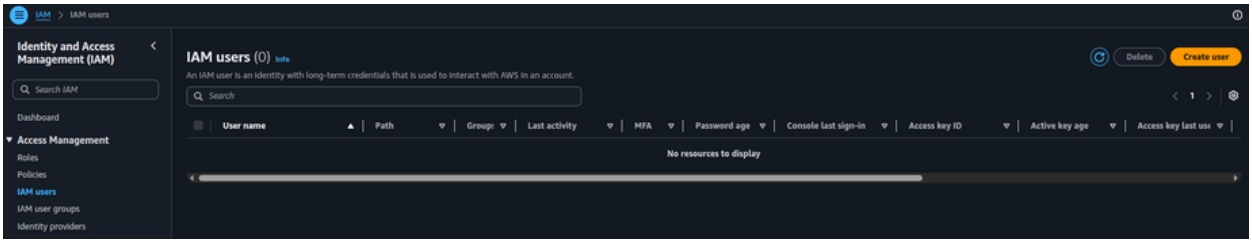


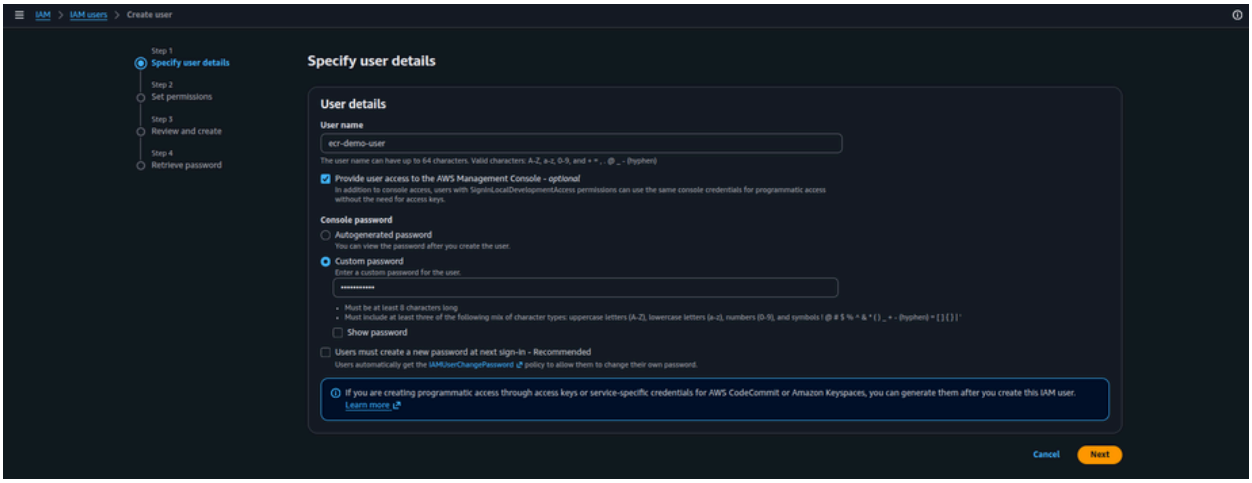
Hands-on lab:

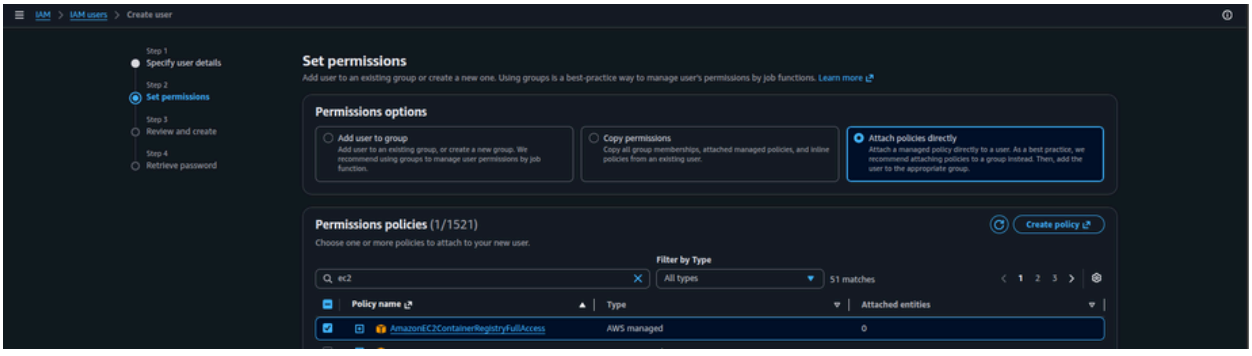
Step 2:

Create a new IAM user with appropriate permission.

To create a new user first sign in to your aws account then search for iam and create new user and keep the credentials for ECR config from your local machine.

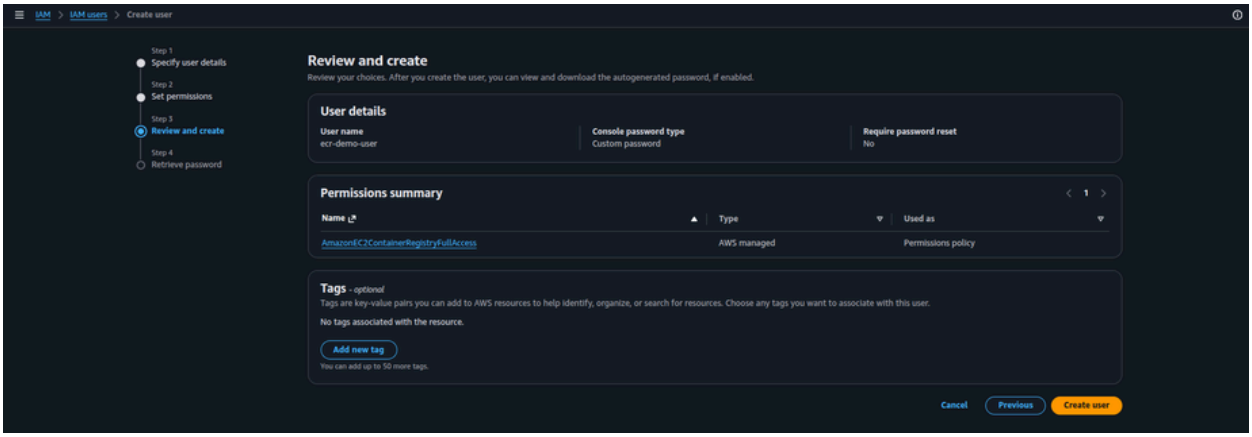






Hands-on lab:

Step 2:

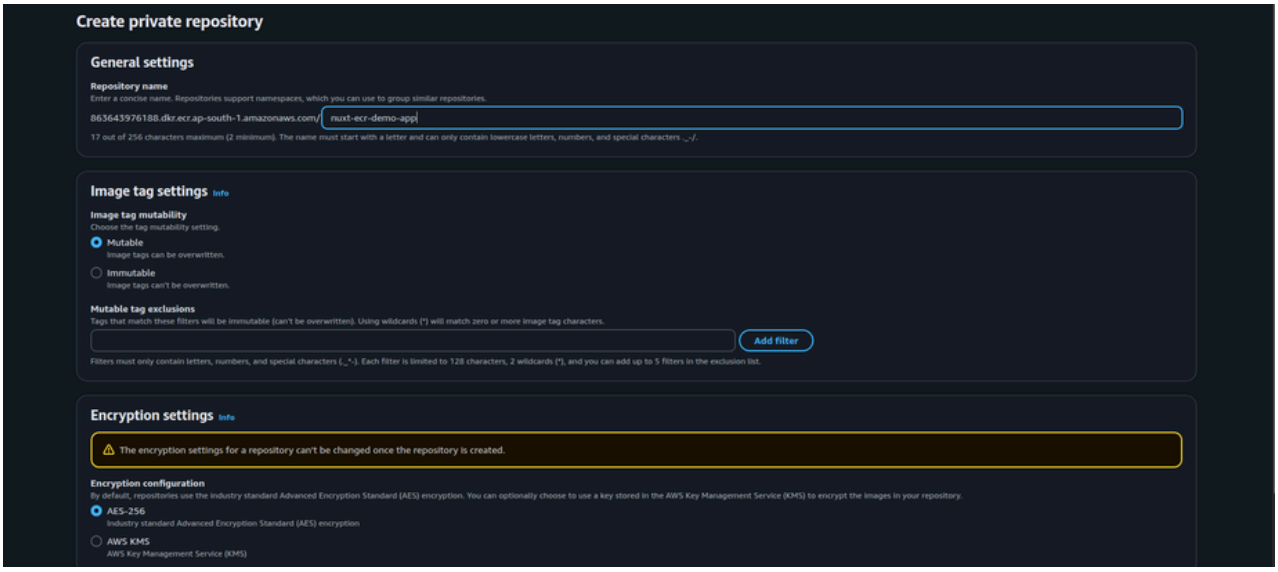


After creating a user successfully. Now create ECR image.

Step 3:

Configure an ECR





Hands-on lab:

After config a new ECR, you have to follow few step to push an image. Here are the example commands.

Authenticate Docker to ECR:

```
aws ecr get-login-password --region us-east-1 | \
    docker login --username AWS --password-stdin <account-id>.dkr.ecr.us-east-1.amazonaws.com
```

Tag, and push a simple image:
Doesn’t need to build again cause already build once.

```
docker tag my-app:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/my-app:v1

docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/my-app:v1
```

Note: I don't have root access to the AWS account, so I'm unable to provide a full process screenshot.

Thank You

Stay Connect:

/in/alamgirweb11

/alamgirweb11

/alamgir.xyz